Liste des prérequis

- Écrire un programme en C avec le HAL STM32CubeL4 (fonctions HAL_GPIO, HAL_UART, HAL_Delay…)
- Utiliser des pointeurs en langage C
- Connaître l'organisation générale d'un projet PlatformIO pour le NUCLEO-L476RG
- Avoir compilé et exécuté au moins un programme sur le NUCLEO-L476RG

SUJET

Énoncé

IL FAUT CHANGER LA PILE !

Toujours chez THS...

L'équipe de supervision a bien apprécié les changements opérés sur l'algorithme de lecture (dans le fichier « Nouveau_Algo_Mesure », désormais porté sur le STM32L476RG) et semble en accord avec la direction prise. Cependant, elle a émis une réserve sur la taille de la pile d'exécution et les temps d'accès aux variables. Le passage de la plateforme Arduino a la carte NUCLEO-L476RG a mis en évidence que la pile par défaut configurée dans le linker script ne laisse que très peu de marge...

"Décidément, notre algorithme ne sera jamais assez performant pour eux ! Nous avons considérablement réduit le nombre d'appels de fonctions et nous n'avons plus qu'un seul tableau ! 2 constantes, 3 variables locales dont une static — je ne vois pas comment on peut faire mieux sur un Cortex-M4 ! Je ne vais quand même pas mettre ma variable static en globale !"

"Peut-être que nous pouvons changer la manière de gérer notre tableau en l'allouant dynamiquement avec malloc() ? Sur le STM32 ça utiliserait le tas plutôt que la pile..."

"Je ne suis pas sûr que ce soit une bonne idée. Il faudrait inclure stdlib.h en plus des bibliothèques HAL des capteurs, et sur un système embarqué la fragmentation du tas est un vrai risque. On peut toujours essayer en surveillant l'état de la pile avec le debugger sous VSCode pendant l'exécution, mais j'ai peur que cela n'engendre d'autres problèmes..."

Ressources pour les étudiants

Netacad : Linux

problèmes...

Ressources pour les étudiants

Netacad : Linux

« Module 13 - Where Data is Stored » en complément.

Netacad : Langage C

Module 6 pour la partie Fonctions.

Zones mémoires

Révisions et compléments sur la mémoire

Organisation de la mémoire (pile)

Programme et assembleur

Lien avec l'assembleur

Gestion de la pile

Taille de la pile

Présentation de la pile (vidéo)

Utiliser la pile

A noter : toutes les vidéos sont intéressantes.

Ressource principale

Nouveau Algo Mesure

Bibliothèques

Bibliothèque Stdlib

CESI
ÉCOLE D'INGÉNIEURS

Nouveau Algo Mesure
THS – Algorithme WWW

THS – ALGORITHME WWW

```
// Constantes
Nb_Capteurs = x
Nb_Val = 10

// Variables locales (loop)
Static ind_moy // initialisée à 0
Struct Capteurs Nb_erreur
Struct Capteurs Moy_gliss[Nb_Val]

// PROCEDURE LECTURE(POINTEUR_TABLEAU , POINTEUR_ERREURS)
Lecture(*Tab_Val, *Erreurs)
Float Mesure, Erreur
Pour chaque capteur i
{
Mesure = 0
Erreur = Lecture_capteur(&Mesure,i) // Pour le moment, une fonction simulera la lecture du capteur à l'aide d'une valeur aléatoire
Si Erreur Alors
        (Erreursài)++
Sinon
        Add_Val(Tab_Val,Mesure)
}
```

NOUVEAU ALGO MESURE

CESI
ÉCOLE D'INGÉNIEURS

- Nouveau Algo Mesure
  - THS – Algorithme WWW

```
// PROCEDURE LECTURE(POINTEUR_TABLEAU , POINTEUR_ERREURS)
Lecture(*Tab_Val, *Erreurs)
Float Mesure, Erreur
Pour chaque capteur i
{
Mesure = 0
Erreur = Lecture_capteur(&Mesure,i) // Pour le moment, une fonction simulera la lecture du capteur à l'aide d'une valeur aléatoire
Si Erreur Alors
        (Erreursài)++
Sinon
        Add_Val(Tab_Val,Mesure)
}
-----------------------------------------------------


// PROCEDURE ADD_VAL(POINTEUR_TABLEAU , VALEUR)
Add_Val(*Tab_Moy,Val)
Tab_Moy[ind_moy] = Val
Si ind_moy >= Nb_Val – 1 Alors
        ind_moy = 0
Sinon
        ind_moy++
-------------------------------------------
```